

데이터가 어디서 와서 어디로 가는가

ping-v2 파이프라인 전체 그림 — 실유저 데이터와 합성 데이터는 어디서 갈라지고 어디서 만나는가

작성 2026-08-04 · 이 문서는 docs/ 위키(2층) 소속이다

확인 방법 추정이 아니라 실제 DB에 질의해서 쓴 수치다. 확인 시점은 2026-08-04

읽는 사람 코드를 직접 쓰지 않고 기획·데이터 판단을 하는 작업자

1. 한 장 요약

저장소가 셋 있고, 데이터 갈래는 둘이다. 갈래 둘은 운영 단계에서 절대 섞이지 않고, BigQuery 에서 처음 이자 마지막으로 만난다.



핵심 한 줄

웹앱은 Supabase 만 본다. 합성 데이터는 웹앱에 구조적으로 들어갈 수 없다 — 다른 서버, 다른 데이터베이스이기 때문이다. 코드로 확인되는 사실이다 (web/src/lib/supabase/client.ts 가 NEXT_PUBLIC_SUPABASE_URL 하나만 읽는다).

2. 저장소 셋 — 각각 무엇이 들었나

저장소	어디에	무엇이 들었나	누가 쓰나
-----	-----	---------	-------

Supabase 실유저	클라우드 (무료 티어)	지인들이 실제로 만든 데이터. 계정·투표·하트·게시글. NEIS 에서 받은 전국 학교 5,725개 마스터도 여기 있다.	웹앱 이 직접 읽고 쓴다
로컬 pgtest 합성	내 PC 의 Docker 컨테이너	생성기가 지어낸 20,000명의 12개월치 활동. Supabase 와 표 구조(40표)가 완전히 같다.	아무도. BigQuery 로 올리기 위한 중간 기착지
BigQuery raw 실 합성	클라우드 (서울 리전)	위 둘을 같은 표에 섞어 쌓는다. 대신 모든 행에 <code>_source</code> 가 붙어 어느 쪽에서 왔는지 알 수 있다.	분석. 앞으로 P6-P7이 여기서 출발

왜 로컬 Postgres 를 한 번 거치나 — CSV 에서 바로 BigQuery 로 가면 안 되나

두 가지 때문이다. 첫째, **정합성 검사 17종**이 SQL 이라 관계형 DB 가 있어야 돈다. "하트를 쓴 기록 없이 힌트를 샀다" 같은 모순은 CSV 를 눈으로 봐서는 못 잡는다.

둘째, **실서비스와 똑같은 제약(FK·UNIQUE·CHECK)을 통과하는지**를 확인해야 한다. 통과하지 못하는 데이터는 "실제로는 일어날 수 없는 일"이 섞여 있다는 뜻이다. BigQuery 는 제약이 없어서 무엇이든 그냥 받아준다 — **틀린 것도 받아준다.**

3. 질문 24개 vs 240개 — 물어보신 사례

같은 `question` 이라는 표가 두 데이터베이스에 각각 따로 있다. 같은 표에 264개가 들어 있는 것이 아니다.

	Supabase 실서비스	로컬 pgtest 합성
질문 수	24개	240개
누가 넣었나	W5 에서 사람이 손으로 고른 24개. 실제 이용자에게 출제된다	생성기가 템플릿에서 뽑아 자동으로 만든 240개
카테고리	8개	8개 (같은 마스터)
외모·신체	카테고리는 켜져 있다(<code>is_active=true</code>). 다만 실제 질문 문장은 아직 안 넣었다	16개 들어 있다 (240개의 6.7%)
id 범위	8 ~ 46	1 ~ 240

그럼 BigQuery 에서 합쳐지면 어떻게 되나

한 표 안에 나란히 공존한다. 덮어쓰지 않는다.

시점	raw.question 의 내용
지금 (실유저만)	24행 — 전부 <code>_source='supabase'</code> , id 8~46
합성을 올린 뒤	264행 — <code>'supabase'</code> 24행(id 8~46) + <code>'local'</code> 240행(id 1~240)

덮어쓰지 않는 이유는 적재가 `_source` 까지 맞춰야 "같은 행"으로 보기 때문이다 (`MERGE ... ON T._source = S._source AND T.id = S.id`). 즉 BigQuery 에서 실질적인 기본키는 `id` 가 아니라 (`_source, id`) 다.

실유저 질문 24개가 전부 합성 질문과 id 가 겹친다

실유저 질문 id 가 8~46 이고 합성이 1~240 이므로, 24개 중 하나도 빠짐없이 같은 번호의 합성 질문이 존재한다.

`JOIN ... USING (id)` 로 조인하면 실유저의 투표에 합성 질문 문장이 붙고 행 수도 늘어난다(1:2 매칭). 그리고 오류가 나지 않는다 — 숫자가 다 맞아떨어지기 때문이다.

올바른 조인은 이렇다:

```
JOIN question q ON q.id = v.question_id AND q._source = v._source
```

여기서 조용한 함정이 생긴다

두 DB 의 질문 id 가 둘 다 1부터 시작한다. 그래서 BigQuery 에 섞이면 실서비스의 3번 질문과 합성의 3번 질문이 같은 값을 갖는다. 서로 다른 문장인데도.

id 로 조인하면 실유저의 투표에 합성 질문이 붙는다. 그리고 오류가 나지 않는다. 숫자가 맞아떨어지기 때문이다. 실제로 `app_user` 16개, `vote_item` 26개의 id 가 겹치는 것이 확인됐다.

그래서 BigQuery 에서는 반드시 `_source` 를 조인 조건에 넣어야 한다. P6 에서 `_source || '-' || id` 형태의 키를 만들어 이 실수를 구조적으로 막을 계획이다.

같은 구조, 다른 내용 — 이게 설계 의도다

합성 데이터는 **실서비스 스키마를 그대로 따른다**. 표 40개, 컬럼 이름, 제약까지 같다. 그래서 합성 데이터로 짠 분석 쿼리를 **실데이터에 그대로 쓸 수 있다**. 스키마를 합성 데이터 편의를 위해 바꾸지 않는다는 것이 결정 사항이다.

4. 지금 적재하고 있는 것은 어디로 가나

오늘 만든 1억 4,126만 행은 **내 PC의 pgtest 컨테이너**로 들어가는 중이다. Supabase 에는 **한 줄도 안 간다**. 웹앱에도 영향이 없다.

순서	단계	무슨 일
1	생성 <code>generate.py</code>	CSV 40개 (7.4GB) 를 만든다. 완료 — 10분 16초
2	적재 <code>load.py</code>	<code>COPY</code> 로 로컬 Postgres 에 붓는다. 완료
3	<code>95_resync_sequences.sql</code>	id 를 직접 지정해 넣으면 자동 번호표가 안 움직인다. 되돌리지 않으면 다음 가입이 id=1 을 발급하려다 충돌한다
4	<code>96_backfill_updated_at.sql</code>	진행 중 . 아래 설명
5	정합성 17종	모순이 없는지. 위반 0 이어야 한다
6	BigQuery 적재	<code>_source='local'</code> 로 올린다

4번이 왜 이렇게 오래 걸리나 (1시간 가까이)

`updated_at` 은 "이 행이 마지막으로 바뀐 시각"이고, BigQuery 증분 적재가 **이 값 하나만 보고** 무엇을 새로 올릴지 정한다.

그런데 이 컬럼의 기본값이 "지금"이라, `COPY` 로 부으면 **1억 4천만 행이 전부 오늘 오후로 찍힌다**. 12개월치가 하루에 뭉쳐서 시간 분석이 전부 무의미해진다.

그래서 각 행의 **원래 시각**(투표한 시각, 가입한 시각...)으로 되돌린다. 1억 4천만 행을 하나씩 다시 쓰는 작업이라 오래 걸린다.

5. 실유저와 합성을 어떻게 구분하나 — 표식이 둘 있다

표식	어디에 있나	쓰임
<code>_source</code>	데이터 표 40개 전부 (적재할 때 붙인다. NOT NULL)	이게 진짜 구분자다. 'supabase' 또는 'local' . 실유저만 보려면 <code>WHERE _source = 'supabase'</code>
<code>is_synthetic</code>	<code>app_user</code> 한 표에만	원래 이걸 쓰려 했으나 39개 표에는 없어서 못 쓴다. id 충돌도 못 푼다. 지금 Supabase 에서 이 값이 true 인 사람은 0명

여기에 적재 시각 `_loaded_at` 도 함께 붙는다. 즉 BigQuery 의 모든 표는 원래 컬럼 + `_source` + `_loaded_at` 구조다.

실제로 확인한 것 (2026-08-04)

BigQuery raw 의 표 41개 중 40개에 `_source` 가 있다. 없는 하나는 `_load_state` 인데, 이건 데이터가 아니라 "어느 표를 어디까지 올렸나"를 적어두는 파이프라인 장부라 예외다. 즉 데이터 표는 빠짐없이 전부 붙어 있다.

왜 아예 다른 데이터셋으로 나누지 않았나

나누면 "실유저와 합성을 같은 쿼리로 비교"하는 것이 불가능해진다. 합성 데이터의 목적이 **실데이터가 적어서 못 보는 것을 대신 보는 것이므로**, 둘을 나란히 놓고 "합성에서 보이는 패턴이 실데이터에도 있나"를 물을 수 있어야 한다.

6. 증분 적재 — 왜 매번 전부 올리지 않나

BigQuery 는 바뀐 것만 올린다. 방식은 이렇다.

1. `_load_state` 라는 표에 "이 표는 여기까지 올렸다"는 시각(워터마크)을 적어 둔다
2. 다음 적재 때 `updated_at > 워터마크` 인 행만 가져온다
3. 올린 뒤 워터마크를 갱신한다

증분은 조용히 틀린다 — 에러가 안 난다

실제로 걸린 함정 넷이 있다. 전부 오류 메시지 없이 잘못된 결과를 냈다.

- 지운 워터마크를 안 지움 — 행만 지우고 워터마크를 남기면, 다음 적재가 "이미 최신"으로 판단하고 아무것도 안 올린 채 조용히 끝난다
- 컬럼 추가 후 워터마크 정지
- 지운 행의 유형 — 원천에서 지워도 BigQuery 에는 남는다(의도된 설계다)
- 과거 시각 백필 — 워터마크보다 과거로 시각을 되돌리면 영원히 안 올라간다

7. 왜 합성 데이터를 만드나 — 실유저가 있는데

실유저가 23명이기 때문이다. 이 규모로는 답할 수 없는 질문이 대부분이다.

물고 싶은 것	실유저 23명으로 가능한가
가입 후 1년간 얼마나 남는가 (잔존)	불가능 — 서비스가 시작된 지 며칠이다
방학·시험기간에 활동이 어떻게 변하나	불가능 — 12개월을 관측해야 한다
하트를 사는 사람과 안 사는 사람이 어떻게 갈리나	불가능 — 표본이 너무 작다
민감한 질문이 실제로 더 신고되나	불가능 — 신고 자체가 거의 없다
파이프라인이 대량 데이터에서 도는가	이건 합성으로만 확인 가능하다

그러나 합성 데이터는 정답이 아니다. 내가 정한 분포대로 나오게 만든 것이므로, "합성에서 이런 결과가 나왔다"는 내가 그렇게 설정했다는 뜻 이상이 아닐 수 있다. 합성으로는 분석 방법과 파이프라인을 검증하고, 사실 판단은 실데이터로 한다.

8. 지금 어디까지 왔고 앞으로 무엇이 남았나

단계	내용	상태
P0~P2	스키마 · 합성 생성 · 로컬 적재	완료
P3	NEIS 학교·급식·학사일정 수집	데이터는 받았고 자동화(DAG)는 남음
P4	BigQuery 적재	완료 — 지금은 실유저만 올라가 있다
P5	품질 검증	다음 차례
P6	stg / mart — 분석용 정리 층	첫 작업은 대리키(id 충돌 방지)
P7	대시보드	P6 이후, raw 에 직접 붙이면 안 된다

P6 을 건너뛰고 대시보드를 붙이면 안 되는 이유 — 돈이다

BigQuery 는 저장이 아니라 읽은 양으로 과금한다. 무료 한도는 월 1 TiB 다.

합성 데이터의 가장 큰 표(vote_candidate)는 4.9 GiB 다. 대시보드가 이 표를 통째로 훑으면 한 번에 무료 한도의 0.5% 가 나간다. 차트 20개가 하루 30번씩 새로고침하면 월 17 TiB 가 되어 십수만원이 청구된다.

P6 은 미리 집계해 둔 작은 표를 만드는 단계다. 대시보드가 작은 표만 보게 하면 이 문제가 사라진다. 그때까지의 임시 방어책으로 일일 쿼리 상한 30 GiB 를 걸어 두었다.

9. 자주 헛갈리는 것

질문	답
합성 데이터가 실서비스에 섞일 수 있나?	없다. 다른 서버·다른 DB 이고, 웹앱은 Supabase 만 본다
내 PC 의 pgtest 를 지우면 큰일 나나?	아니다. CSV 에서 다시 부으면 된다. 다만 BigQuery 에 이미 올렸다면 워터마크도 함께 지워야 한다
실유저 23명 데이터는 안전한가?	Supabase 에 있고 합성 작업이 건드리지 않는다. BigQuery 재적재도 _source='local' 만 지운다
웹앱에 나오는 급식·학교는 어디 것인가?	NEIS 실제 데이터다(서울고 공개 데이터를 빌려 쓴다). 합성이 아니다
질문 240개가 실제 이용자에게 나가나?	아니다. 실서비스는 Supabase 의 24개만 쓴다
합성 데이터에 진짜 사람 정보가 있나?	없다. 닉네임까지 전부 조합해서 만든다. 애초에 이 서비스는 실명·이메일·전화번호를 받지 않는다

ping-v2 · 이 문서는 실제 DB 질의로 확인한 2026-08-04 시점의 사실이다. 수치는 시간이 지나면 달라진다 — 구조와 이유는 유지된다.